

OPERATING SYSTEMS AND SYSTEM PROGRAMMING · 25CS2104E

ExecGuard

Policy-Based Linux Executable Integrity and Write Protection

Project Report

Submitted by

Hrushika Habeeb (2520090054)

Guided by

Raghupathi M

Date

October 2026

Abstract

On a standard Linux system, any sufficiently privileged process can overwrite, truncate, replace or delete a system executable. File permissions express *who* may write a file; they cannot express the intent “this binary must not change except through the package manager.” ExecGuard adds that missing control. It is a policy-driven integrity mechanism that attaches eBPF programs to four Linux Security Module (LSM) hooks and denies, before the operation commits, every write-intent open, truncation, deletion and rename that targets a protected executable, unless the caller is an explicitly trusted updater, a time-bounded maintenance window is open, or the system is in audit-only mode. Protected files are identified in the kernel by their (device, inode) identity rather than by path, which makes enforcement an $O(1)$ hash-map lookup and closes the hard-link, symlink and rename aliasing that defeats path-based tools. A userspace daemon translates a line-oriented policy into kernel maps, streams every decision to a structured audit log, and exposes a control CLI; a SHA-256 baseline provides a secondary, detective layer for tampering that happens while the mechanism is not loaded. This report describes the design, implementation, verification strategy and limitations of the system. The userspace logic and compiled kernel object pass all 63 automated suite checks and 71 core assertions; a 29-case catalogue specifies the in-kernel enforcement tests to be executed on a BPF-LSM host.

1. Introduction

1.1 Problem statement

Executable integrity is a precondition for trusting anything a system does. An attacker who can replace `/usr/sbin/sshd`, `/bin/bash` or a security agent’s binary gains persistence and can subvert every subsequent action on the host. The standard Unix discretionary access-control model offers no defence here once the attacker holds root, or once a misconfigured service runs with sufficient privilege: root may write any file.

Existing integrity tools are predominantly *detective*. AIDE and Tripwire hash files periodically and report differences after the fact; by the time a scan runs, the tampered binary may already have executed. What is missing is a *preventive* control that (a) blocks modification at the moment it is attempted, (b) still permits legitimate updates, and (c) records every attempt.

1.2 Objectives

1. Prevent modification, truncation, replacement and deletion of executables designated by policy, **before** the kernel commits the operation.
2. Permit legitimate updates through narrowly identified trusted updaters and a time-bounded maintenance window.
3. Support an audit-only mode for safe rollout.
4. Produce a structured, per-decision audit trail.
5. Provide an integrity baseline to detect offline tampering.
6. Run on a stock distribution kernel without kernel patches or modules.

1.3 Scope

ExecGuard protects the on-disk contents and continued existence of specific files accessed through the virtual filesystem (VFS). It is a runtime control; it does not claim to defend against an attacker who can unload it, boot a different kernel, or edit the disk offline. These boundaries are stated in Section 8 and in `docs/threat-model.md`.

2. Background

2.1 Linux Security Modules

The LSM framework places security hooks at mediation points throughout the kernel. A hook is called inside the system-call path after arguments are resolved but before the operation takes effect. Its return value is the decision: zero allows the operation, and a negative `errno` aborts it and is returned to the caller. SELinux, AppArmor, Yama and Landlock are all built on this framework.

2.2 eBPF and BPF-LSM

eBPF allows verified programs to be loaded into a running kernel. Since Linux 5.7, the BPF-LSM facility (also known as Kernel Runtime Security Instrumentation, KRSI) allows such programs to attach to LSM hooks. The kernel's verifier guarantees that a loaded program terminates and accesses memory safely, so security logic can be added without writing or loading a kernel module. BPF-LSM requires `CONFIG_BPF_LSM=y` and `bpf` in the active LSM list.

2.3 CO-RE

Compile Once, Run Everywhere (CO-RE) uses the kernel's BPF Type Format (BTF) metadata to relocate structure field accesses at load time. ExecGuard reads fields such as `inode->i_ino` and `task->mm->exe_file` through `BPF_CORE_READ`, so one compiled object works across kernel builds whose internal structure layouts differ.

2.4 Inodes and file identity

A path is a directory entry; the file itself is the inode it points to. Several paths (hard links) can name one inode, and a rename changes the name without changing the inode. Any control keyed on path strings must resolve every alias; a control keyed on inode identity protects the object directly.

3. Comparison with existing approaches

Approach	Blocks before modification?	Blocks delete/rename?	Allows authorised updates?	Survives root?	Notes
File permissions / ownership	Only for non-owners	Via directory permissions	Yes	No	Root bypasses entirely
<code>chattr +i</code> (immutable flag)	Yes	Yes	No (must clear flag first)	No	Root with <code>CAP_LINUX_IMMUTABLE</code> clears it; no per-updater exception
AIDE / Tripwire	No (detective)	No (detective)	n/a	Partially	Detects changes at next scan
fanotify permission events	Write-open only	No (notification only)	Yes (userspace logic)	No	Rename and unlink cannot be denied
SELinux / AppArmor policy	Yes	Yes	Yes (via domains/profiles)	Yes, with policy	Powerful but complex to author for this single goal
IMA/EVM appraisal	No (blocks execution of modified files)	No	Yes (re-signing)	Yes, with Secure Boot	Complementary to ExecGuard
ExecGuard (BPF-LSM)	Yes	Yes	Yes (by updater inode)	No (documented)	One-purpose policy; no kernel rebuild

ExecGuard occupies a specific niche: a small, auditable, single-purpose preventive control that is simpler to deploy than a full mandatory access-control policy, stronger than detective scanners, and that distinguishes legitimate updaters from everything else.

4. System design

4.1 Architecture

ExecGuard has two planes. The **kernel plane** consists of four BPF-LSM programs and four BPF maps. It holds all enforcement logic, so enforcement does not depend on userspace being responsive. The **userspace plane** consists of one multi-call binary that acts as the daemon (`execguardd`) or the CLI (`egctl`) depending on the name it is invoked by, a separate trusted-updater helper (`eg-updater`), and an optional read-only dashboard.

Component	Location	Responsibility
LSM programs	src/kernel/execguard.bpf.c	Mediate <code>file_open</code> , <code>inode_unlink</code> , <code>inode_rename</code> , <code>inode_setattr</code> ; decide; emit events
Shared ABI	src/include/execguard/eg_common.h	Fixed-width structs and enums exchanged through maps and the ring buffer
Daemon	src/userspace/execguard.c	Load/attach BPF, resolve policy to inodes, populate maps, drain events to <code>audit.jsonl</code> , serve the control socket
CLI (<code>egctl</code>)	src/userspace/execguard.c	Status, runtime protect/trust, maintenance, policy validation, integrity, logs
<code>eg-updater</code>	src/tools/eg-updater.c	Minimal ELF whose inode can be trusted; used by demos and tests
Dashboard	src/dashboard/	Read-only HTTP view of status and recent events, bound to localhost

4.2 Kernel data structures

Map	Type	Key → Value	Capacity	Purpose
<code>protected_inodes</code>	hash	<code>(dev, ino) → flags</code>	65,536	Set of protected files
<code>trusted_exes</code>	hash	<code>(dev, ino) → 1</code>	4,096	Executables allowed to modify protected files
<code>state</code>	array	<code>0 → {enforcing, maintenance_until_ns}</code>	1	Global mode and maintenance deadline
<code>events</code>	ring buffer	<code>struct eg_event</code>	256 KiB	Decision stream to the daemon

The device number in every key uses the kernel's `dev_t` encoding, `(major << 20) | minor`, which is what BPF reads from `inode->i_sb->s_dev`. glibc's `st_dev` uses a different layout, so userspace re-encodes it in a single audited function, `eg_encode_kdev()`. An error here would cause every map lookup to miss silently; it is covered by dedicated unit tests.

4.3 Mediated operations

Attack vector	System call(s)	LSM hook	Mechanism
Overwrite / append / truncate-at-open	<code>open(O_WRONLY O_APPEND O_TRUNC)</code>	<code>file_open</code>	Deny any open with <code>FMODE_WRITE</code> on a protected inode
Truncate by path or descriptor	<code>truncate</code> , <code>ftruncate</code>	<code>inode_setattr</code>	Deny <code>ATTR_SIZE</code> changes on a protected inode
Delete	<code>unlink</code> , <code>unlinkat</code>	<code>inode_unlink</code>	Deny unlink of a protected inode
Replace via rename	<code>rename</code> , <code>renameat2</code>	<code>inode_rename</code>	Deny rename over a protected inode, and rename of a protected inode away
Writable memory mapping	<code>mmap(PROT_WRITE, MAP_SHARED)</code>	<code>file_open</code>	Requires a writable descriptor, which is never granted

Blocking at `file_open` is deliberately broad: a process cannot `write`, `pwrite`, `ftruncate` or create a writable shared mapping without first holding a writable descriptor, so one decision closes all of those paths.

4.4 Decision procedure

For each mediated operation the kernel program:

1. Converts the target inode to a `(dev, ino)` key and looks it up in `protected_inodes`. If absent, it returns 0 immediately and logs nothing. Unprotected files therefore pay only the cost of one hash lookup.
2. Identifies the caller by the inode of its executable image, `current->mm->exe_file`. If that key is in `trusted_exes`: **allow**, reason *trusted updater*.
3. Otherwise, if the monotonic clock is before `maintenance_until_ns`: **allow**, reason *maintenance window*.
4. Otherwise, if `enforcing == 0`: **allow but record as AUDIT**, reason *audit-only mode*.
5. Otherwise: **deny** with `-EPERM`, reason *protected executable*.

Every decision on a protected file emits one ring-buffer event. If the buffer is full, the event is dropped but the decision is not; the security outcome never depends on logging capacity.

4.5 Trusted-updater identity

A trusted updater is identified by the inode of the executable it is running, read in-kernel. This is why `eg-updater` is a compiled ELF rather than a shell script: a running script's executable image is its interpreter, so trusting a script would trust `/usr/bin/bash` for every process on the system. Identity by process name (`comm`) was rejected because any process can set its own name.

4.6 Maintenance window

`egctl maintenance enable --duration 5m` stores a deadline against `CLOCK_MONOTONIC`. The kernel compares `bpf_ktime_get_ns()` with that deadline on every decision, so the window expires on its own without any userspace action and cannot be extended by changing the wall clock.

4.7 Policy language

Directive	Meaning
<code>PROTECT <path-or-glob></code>	Protect a file, or every executable matching a glob (expanded at load time)
<code>TRUST <executable></code>	Allow this ELF, by inode, to modify protected files
<code>MODE enforce audit</code>	Global default; the secure default when unspecified is <code>enforce</code>
<code>DENY user=<name></code>	Reserved for future userspace enforcement

A path matched by a glob is protected only if it is an executable: it has an execute bit, begins with the ELF magic, or begins with `#!`.

5. Implementation

Part	Language	Lines
Kernel programs	C (BPF target)	257
Shared ABI header	C	119
Daemon and CLI	C	1,535
Trusted updater	C	48
Scripts, tests, benchmark	Bash	2,015
Core unit tests	C	415
Dashboard	Python, HTML	215

The build (`make`) generates `vmlinux.h` from the build host's BTF, compiles the BPF object with clang for the BPF target, generates a libbpf skeleton with `bpftool gen skeleton`, and compiles the userspace binary against it. All generated output is confined to `build/`. On the reference build host the complete build finishes in about one second with zero compiler warnings.

Two implementation details deserve mention. First, the ABI structs carry `_Static_assert` size and offset pins in the shared header, so both the BPF compiler and the host compiler refuse to build if the layout of a struct that crosses the kernel/user boundary ever changes. Second, the `inode_setattr` hook signature is kernel-version-sensitive (it gained a leading `struct mnt_idmap *` argument in 6.3); the code targets Linux 6.x and the required change for older kernels is documented.

6. Verification

6.1 Strategy

Correctness is verified at two levels, matching what different hosts can run.

- **Level 1, userspace and build artifacts.** Runs on any host that can build the project, without root. It establishes that the policy engine, device encoding, integrity subsystem and CLI behave as specified, and that the compiled BPF object contains the expected programs, maps, license and ABI.
- **Level 2, in-kernel enforcement.** Requires root, a BPF-LSM kernel and a running daemon. Each case attempts a real operation and checks whether the kernel allowed or denied it. Suites that cannot run exit with status 77 and are reported as skipped, never as passed.

6.2 Level 1 results

Executed on 7 October 2026 on Ubuntu 24.04.5, kernel 6.18, clang 18.1.3, libbpf 1.3.0 (run results/cloud-build-container-20261007T030825Z).

The suite has two layers. `test_core.c` links directly against the userspace source and makes 71 assertions; `test_userspace.sh` treats that program as one check and adds 62 black-box checks of the built binaries and BPF object.

Core assertions (<code>test_core.c</code>)	Count	Result
ABI layout of structs shared with the BPF program	7	Pass
Kernel device encoding and inode resolution	8	Pass
SHA-256 against FIPS 180-2 vectors (incl. one million bytes)	5	Pass
Executable detection	6	Pass
JSON escaping and parsing	5	Pass
Audit-log enum rendering	3	Pass
Maintenance-duration parsing	8	Pass
Policy loading, validation and line parsing	14	Pass
Glob expansion over the fixture tree	5	Pass
Integrity baseline, scan and verify	7	Pass
Daemon path mirror idempotence on reload	3	Pass
Total	71	71 / 71 pass

Suite checks (test_userspace.sh)	Count	Result
Core library (the 71 assertions above, as one check)	1	Pass
egctl exit codes, policy validation and rejection, daemon-unreachable handling	24	Pass
Integrity workflow through the CLI (baseline, schema, tamper, delete)	13	Pass
Audit-log tail	5	Pass
eg-updater behaviour	5	Pass
Compiled BPF object: LSM sections, license, maps, BTF, ABI sizes	15	Pass
Total	63	63 / 63 pass

6.3 Level 2 test catalogue

The 29 enforcement cases are specified in `data/test-cases/attack-matrix.csv`. Each row names the technique, the syscalls it issues, the LSM hook that mediates it, and the expected decision and audit reason.

Suite	Cases	Expected outcome
Functional lifecycle	6	Writes allowed before protection and after unprotection, denied while protected; reads and execution always allowed
Security matrix	13	Untrusted: all five mutation vectors denied. Unprotected: allowed. Trusted updater: allowed. Maintenance: allowed, then denied after disable. Audit-only: allowed, then denied after re-enabling enforcement
Bypass attempts	10	Shell redirect, Python <code>open/os.open</code> , <code>dd</code> , <code>truncate</code> , <code>sed -i</code> , temp-file rename-over, delete, write through a hard-link alias, and <code>chmod 0777</code> followed by a write: all denied

The hard-link case is the distinguishing one: a second name for the protected inode is created and written to. A path-based control would permit this; an inode-keyed control must deny it.

These cases require a BPF-LSM host. On the container used for the Level 1 run, the kernel reports `CONFIG_BPF_LSM=y`, but the container runtime denies `BPF_PROG_LOAD` for LSM programs even to root (the captured `daemon.log` shows `Operation not permitted`), so all three suites were correctly recorded as skipped. The Level 2 run is executed in an Ubuntu 24.04 Multipass VM

with `bpf` added to the LSM list, following `docs/build-and-run.md`, using `sudo make results`. That run additionally captures every kernel audit event and the benchmark.

6.4 Performance methodology

`src/benchmarks/bench.sh` measures, on the host where it runs: open-for-read latency on a protected versus an unprotected file (isolating the cost of the hook and one map lookup on the allow path), latency of a denied open-for-write, and integrity-scan wall time. It prints no pre-computed figures. By construction, unprotected files return from the hook after a single failed hash lookup and emit no event; protected files add one further lookup in `trusted_exes`, one array lookup and one ring-buffer reservation per mediated operation.

6.5 Defects found and corrected during verification

#	Defect	Effect	Correction
1	<code>install.sh</code> read <code>\$?</code> inside <code>if ! check-env.sh; then</code>	The negated status is always 0, so an <code>UNSUPPORTED</code> host was never rejected	Exit code captured explicitly before branching
2	<code>check-env.sh</code> tested <code>ls /usr/include/...</code> <code>/usr/local/include/...</code>	Fails when either path is absent, so a normal installation reported “libbpf headers not found”	Test each location independently
3	Daemon path mirror appended on every <code>PROTECT/RELOAD</code>	<code>egctl status</code> protected-file count and <code>policy list</code> duplicated on each reload	Mirror insertion made idempotent; unit test added
4	<code>struct eg_event</code> absent from the BPF object’s BTF	Its kernel/user layout agreement was unverifiable by inspecting the object	<code>_Static_assert</code> pins in the shared header, enforced by both compilers

7. Mapping to operating-systems concepts

ExecGuard element	Concept
LSM hook programs	Reference monitor; complete mediation
Hooks inside <code>open</code> , <code>unlink</code> , <code>rename</code> , <code>setattr</code>	System-call interposition at the user/kernel boundary
<code>(device, inode)</code> keys	The inode abstraction; identity versus naming
<code>protected_inodes</code> and <code>trusted_exes</code>	Access-control matrix of subjects and objects
Trust by executable inode	Principals and the confused-deputy problem
Daemon capabilities only (<code>CAP_BPF</code> , <code>CAP_SYS_ADMIN</code>)	Least privilege and privilege separation
Monotonic maintenance deadline	Time-bounded capabilities; monotonic versus real-time clocks
Ring buffer to audit log	Kernel-to-user event streaming; accounting
<code>-EPERM</code> returned from the hook	Error propagation across the protection boundary

`docs/os-concept-mapping.md` discusses each in detail.

8. Limitations

1. **Not self-protecting against root.** Root can stop the daemon or detach the programs. Defending the mechanism requires IMA/EVM, Secure Boot and kernel lockdown, which are out of scope.
2. **Offline tampering and alternate kernels** bypass any runtime control. The SHA-256 baseline detects the former at the next scan.
3. **Identity is bound at load time.** A new file created at a protected path (for example after protection was removed) has a new inode and is unprotected until the policy is re-applied.
4. **Package-manager upgrades re-key protected files.** `dpkg` writes a new file and renames it over the old path. ExecGuard correctly permits this for a trusted updater, but the upgraded binary is a new inode and remains unprotected until `egctl reload`. The recommended mitigation is an APT `DPkg::Post-Invoke` hook that runs `egctl reload`.
5. **Confused deputy.** A trusted updater invoked with attacker-chosen input acts with full trust. The trusted set must be minimal.
6. **Kernel and environment dependence.** BPF-LSM must be compiled in and active, and container runtimes typically prevent loading LSM programs.
7. **Scope.** In-memory attacks, execution from unprotected locations, and kernel compromise are not addressed.

9. Future work

- Automatic re-keying: observe `ALLOW/RENAME` events from trusted updaters and re-resolve the affected path, removing the need for a post-upgrade reload (limitation 4).
- Directory watching with fanotify to protect newly created files that match a `PROTECT` glob.
- Per-user `DENY` rules enforced in-kernel using the caller's UID.

- Signed policy files and pinning of the BPF links so that the daemon cannot be silently replaced.
- Integration with IMA appraisal so that a binary modified offline also fails to execute.

10. Conclusion

ExecGuard demonstrates that a narrowly scoped, preventive integrity control can be built on a stock Linux kernel with BPF-LSM, without kernel modules or patches. Its central design choice, identifying files and updaters by inode rather than by name, makes enforcement both constant-time and resistant to the aliasing techniques that defeat path-based controls. The userspace logic and compiled kernel object are verified by an automated suite that passes in full, the enforcement behaviour is specified case by case for execution on a BPF-LSM host, and the system's boundaries are documented as explicitly as its guarantees.

References

1. Linux kernel documentation, "LSM BPF Programs." https://docs.kernel.org/bpf/prog_lsm.html
2. Linux kernel documentation, "Linux Security Module Development." <https://docs.kernel.org/security/lsm-development.html>
3. Linux kernel documentation, "BPF Type Format (BTF)." <https://docs.kernel.org/bpf/btf.html>
4. libbpf documentation. <https://libbpf.readthedocs.io/>
5. A. Nakryiko, "BPF CO-RE reference guide." <https://nakryiko.com/posts/bpf-core-reference-guide/>
6. J. Edge, "Kernel runtime security instrumentation," LWN.net, 4 September 2019. <https://lwn.net/Articles/798157/>
7. fanotify(7), Linux manual page. <https://man7.org/linux/man-pages/man7/fanotify.7.html>
8. Linux Integrity Measurement Architecture (IMA) project. <https://sourceforge.net/p/linux-ima/wiki/Home/>
9. AIDE: Advanced Intrusion Detection Environment. <https://aide.github.io/>
10. J. P. Anderson, *Computer Security Technology Planning Study*, ESD-TR-73-51, U.S. Air Force, 1972 (origin of the reference-monitor concept).

Appendix A. Command reference

```
make                                # build into build/
make test-unit                      # Level 1 tests
sudo make install                   # install binaries, config, systemd unit
sudo systemctl enable --now execguard
egctl status
sudo egctl protect /usr/local/bin/myservice
sudo egctl trust /usr/bin/dpkg
sudo egctl maintenance enable --duration 5m
egctl logs -n 20
sudo egctl integrity baseline && egctl integrity scan
sudo make results                   # full evidence run into results/
```

Appendix B. Repository layout

data/	policies, fixtures, test-case catalogue, schemas
docs/	architecture, design decisions, security and threat models, limitations, OS-concept mapping, build guide, testing guide
reports/	this report
results/	captured evidence, one folder per run
src/	kernel/, include/, userspace/, tools/, config/, systemd/, scripts/, tests/, benchmarks/, dashboard/
Makefile	build, test, results and install entry points